

THE EFFICIENT-AI RESEARCH TRACK

Month 3 — The Efficiency Frontier

Data curation → Distillation → Quantization & LoRA → your capstone model.
Days 61–90, self-contained.

THIS IS THE MONTH THE TRACK WAS NAMED AFTER

Months 1–2 built the machine and the understanding. This month attacks the cost — with your own tiny GPT as the lab rat. Four campaigns: feed it better (data), teach it from a teacher (distillation), store it cheaper (quantization + low-rank), and finally: your own domain model, built under a compute budget, written up like research. Your two standing hypotheses from the roadmap get tested [HERE](#).

Prerequisites: Months 1–2 — especially ai_13 (tiny GPT), ai_14 (training loop), and your prune/quantize scripts from Month-1 Week 3.

WEEK 9 — Data: Better Food Beats More Food

Goal: test YOUR standing hypothesis — raw vs. cleaned data, same model, measured difference. Data curation is the cheapest efficiency lever in existence: it costs no extra compute at training time at all.

9.1 Why data quality is an efficiency technique

Your Month-2 scaling study showed quality costs parameters and steps. But every training step is only as good as the example in it. A duplicated paragraph teaches nothing new the second time — you PAID compute for zero learning. A garbled line actively teaches garbage. So cleaning data raises quality-per-multiply without touching the model. That's why it's the first campaign, and why frontier labs guard their data pipelines more jealously than their architectures.

WHY / FAILURE / MEASURE

Why: the model is a mirror of its diet (you saw this in Month-1 Week 4 ex.1). **Failure:** over-cleaning — filter too hard and you delete the diversity the model needed; dedup too aggressively and you remove natural, useful repetition. **Measure:** ALWAYS the same: two models, identical everything except data, validation loss on a CLEAN held-out set.

9.2 The three basic cleaners (build each, readable)

1. Exact deduplication — your Stage-0 set trick, graduated. Hash every chunk; seen it before? drop it.

```
# ai_15_data.py -- dedup by chunk
def dedup(text, chunk=200):
    seen, kept = set(), []
    for i in range(0, len(text), chunk):
        piece = text[i:i+chunk]
        h = hash(piece.strip().lower())
        if h not in seen:
            seen.add(h); kept.append(piece)
    return "".join(kept)
```

2. Near-duplicate detection — the sneaky ones differ by a word. Cheap trick: hash overlapping word 5-grams; two chunks sharing >80% of their 5-gram hashes are near-dupes, keep one. (This is the working core of MinHash, the industrial method — yours is the readable version.)

3. Quality filtering — drop chunks that are mostly symbols, absurdly short/long lines, or have very low letter-ratio. Three `if` statements. Crude, honest, effective.

THE HONEST CONTROL

Cleaning SHRINKS the dataset — so a fair comparison must separate "cleaner" from "just smaller." Run three, not two: (a) raw full data, (b) cleaned data, (c) raw data randomly cut to the SAME size as (b). If cleaned beats size-matched-raw, quality did it. This control is what makes it research instead of vibes.

9.3 The experiment: your Module-6 hypothesis, for real

1. Assemble a messy corpus on purpose: your text + a public-domain book + paste some sections twice + a little junk. ~200k+ characters.
2. Build the three variants: raw / cleaned / size-matched raw.
3. Train your tiny GPT identically on each (same seed, same steps — Month-2 recipe).
4. Report validation loss (on a clean, held-out slice that none of them trained on) + a side-by-side page of generated samples.

Typical finding: dedup alone gives a real, visible gain — duplicated data wastes steps AND encourages parroting (the model memorizes what it saw twice). Whatever you find: numbers in the log, prediction-first as always.

Week 9 exercises

1. Measure the parroting directly: take 50 random 30-char snippets of generated text; what fraction appear VERBATIM in training data? Compare raw-trained vs dedup-trained. (You've just built a memorization metric.)
2. Sweep dedup chunk size 50 / 200 / 1000. Which over-deletes? Which misses dupes? Where's the sweet spot for your corpus?
3. Write the one-paragraph conclusion of your data study as if for a paper abstract: claim, evidence, caveat.

ANSWERS

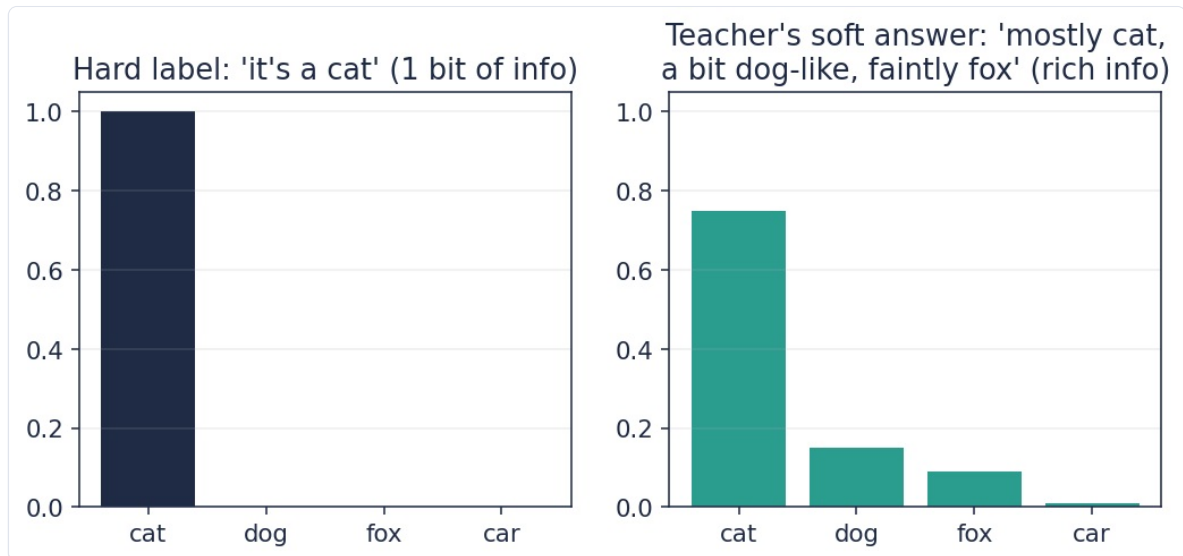
1: raw-trained parrots noticeably more; the metric you built is a real one (verbatim-recall rate). **2:** small chunks over-delete common phrases (natural repetition!); huge chunks miss partial dupes; middle wins. **3:** the exercise IS the answer — claim/evidence/caveat is the skeleton of every abstract you'll ever write.

WEEK 10 — Distillation: A Teacher for Your Student

Goal: the crown jewel of your track. Train a SMALL model to match a BIG one's answers — and measure how much teacher-knowledge survives the shrinking.

10.1 The idea: soft answers carry more truth

Ordinary training shows the student hard labels: "the next char is 'e', period." But your trained BIG model knows something richer: "after 'th', it's 70% 'e', 20% 'a', 8% 'i', almost never 'z'." That whole probability SPREAD — how the possibilities rank, what's almost-right — is knowledge the hard label throws away. Distillation trains the student to match the teacher's full spread.



Hard label vs the teacher's soft answer. The soft answer encodes similarity structure — "dog is a near-miss for cat, car is not" — for free, in every single example.

Why that helps so much: the ranking information acts like thousands of extra mini-labels per example. The student learns not just WHAT but HOW SURE and WHAT'S CLOSE — the teacher's hard-won world-shape, compressed into targets.

10.2 Measuring "how different are two spreads": KL divergence

The math your Stage-1 guide promised would pay off — here's the payment. **KL divergence** measures how badly a student's probability spread misses a teacher's. Hand-trace, 3 options:

```
teacher T = [0.7, 0.2, 0.1]      student S = [0.5, 0.3, 0.2]
KL(T || S) = sum over options of: T_i x log(T_i / S_i)
= 0.7xlog(0.7/0.5) + 0.2xlog(0.2/0.3) + 0.1xlog(0.1/0.2)
= 0.7x(0.336) + 0.2x(-0.405) + 0.1x(-0.693)
= 0.235 - 0.081 - 0.069 = 0.085      (small: spreads are close)

now a bad student S = [0.1, 0.1, 0.8]:
= 0.7xlog(7) + 0.2xlog(2) + 0.1xlog(0.125) = 1.362 + 0.139 - 0.208 = 1.293  (large!)
```

Properties worth noticing: it's 0 only when the spreads match exactly; and it punishes the student hardest for giving LOW probability where the teacher is confident (that $0.7 \times \log(7)$ term dominates). "Don't dismiss what the teacher is sure of" — encoded in algebra.

The full distillation loss mixes two teachers: $\text{loss} = \alpha \cdot \text{KL}(\text{teacher_spread}, \text{student_spread}) + (1-\alpha) \cdot \text{cross_entropy}(\text{true_label}, \text{student})$. The real data keeps the student honest (teachers make mistakes); the KL term transfers the world-shape. Start $\alpha = 0.5$. One more practical dial: soften both spreads with a temperature ~ 2 before comparing (spreads reveal more ranking structure when flattened slightly — your Month-1 temperature knob, third job).

10.3 The experiment

1. **Teacher:** your best Month-2 GPT (biggest D you trained). Freeze it.
2. **Student:** half the size or less (e.g. D=8 or 16, 1 block).
3. Train THREE students, identical size and steps: (a) hard labels only, (b) distilled ($\alpha=0.5$, temp 2), (c) distilled from teacher's outputs ONLY ($\alpha=1$ — no true labels at all!).
4. Report validation loss + generated samples for all three, plus the teacher.

```
# ai_16_distill.py -- the heart of it
t_scores = teacher_forward(x)          # frozen -- no blame flows here
s_scores = student_forward(x)
T = 2.0
p_teacher = softmax(t_scores / T)
p_student = softmax(s_scores / T)
kl = np.sum(p_teacher * np.log(p_teacher / (p_student + 1e-9)))
loss = 0.5 * kl * T*T + 0.5 * cross_entropy(s_scores, target)
# (the T*T rescales KL blame to match CE blame -- standard bookkeeping)
```

Typical finding — and it should feel slightly magical when your own numbers show it: **student (b) beats student (a) at identical size and compute**. Same brain, better teacher, better model. That gap is knowledge transfer, measured by you. Log how much of the teacher's quality the half-size student kept — that percentage is the central number of your whole research direction.

EFFICIENCY LENS

This is the industrial pattern behind most small high-quality models you can run locally: pretrain big once, distill into small many times. Your Stage-5 dream — "domain model, small, cheap, near-big-quality" — is built on exactly the loop you just wrote.

Week 10 exercises

1. Sweep $\alpha = 0 / 0.5 / 1.0$. Where's the best validation loss? What does $\alpha=1$'s result tell you about how much signal lives in the teacher alone?
2. Distill into a student the SAME size as the teacher. Any gain? (Self-distillation sometimes helps — a genuine research puzzle; whatever you find is a real observation.)
3. Hand-compute $KL(T||S)$ for $T=[0.9,0.1]$, $S=[0.6,0.4]$. Then swap: $KL(S||T)$. Same number? What does the asymmetry mean about who's the reference?

ANSWERS

1: usually mid- α wins; $\alpha=1$ often surprisingly strong — teachers carry most of the needed signal. **2:** report what you see; small gains are documented in the literature, absence at tiny scale is also normal. **3:** $0.9\log(1.5)+0.1\log(0.25)=0.226$; swapped: $0.6\log(0.667)+0.4\log(4)=0.311$ — not symmetric; the first argument is "the truth being missed."

WEEK 11 — Store It Cheaper: Quantization Round 2 + the LoRA Idea

Goal: your Month-1 compression tools meet the transformer — plus the low-rank trick, hand-traced, that lets giant models be adapted by training 1% of them.

11.1 Quantize the GPT (and find who's fragile)

Your Month-1 `quantize()` works unchanged — weights are weights. New question worth research-grade attention: **which parts of a transformer tolerate coarse storage?** Quantize one component at a time — E only / attention Wq,Wk,Wv only / MLP only / final matrix only — at 8, 4, 3 bits, measuring validation loss each time. Build the sensitivity table. (Common finding at all scales: some parts shrug off 4-bit, others crack — embeddings and final layers are often the divas. Yours may differ: that's the point of measuring.) This per-part sensitivity mapping is precisely what serious quantization work does; you're doing the honest miniature.

11.2 The low-rank idea (the matrix trick behind LoRA)

Setup: adapting a trained model to a new domain nudges its big matrices — but storing a full nudge for a 64x64 matrix costs 4096 numbers. Observation: most useful nudges are SIMPLE — they don't need 4096 degrees of freedom. Simple changes can be written as a tall-skinny times short-wide product:

The low-rank idea behind LoRA: two skinny matrices stand in for one big one



Instead of learning a full update to W, learn A (64x4) and B (4x64): 512 numbers approximating a 4096-number change. The "4" is the rank — the update's complexity budget.

Hand-feel with tiny numbers -- a rank-1 update to a 3x3 matrix:

```
A = [1]      B = [2  0  1]
    [0]
    [2]
A @ B = [2  0  1]      9 numbers of update...
        [0  0  0]
        [4  0  2]      ...specified by only 3 + 3 = 6. And structured: row 3 = 2 x row 1.
```

LoRA = freeze the trained W entirely; learn only A and B; use $W + A@B$ at runtime. Blame flows only into A and B — tiny memory, tiny optimizer state, and you can keep many little (A,B) "adapters" for different domains atop one frozen model.

```
# ai_l17_lora.py -- adapt your GPT to a NEW text domain, three ways
# (a) full fine-tune: unfreeze everything          (baseline quality, max cost)
# (b) LoRA rank 4 on attention matrices only:
out = x @ (Wq + Aq @ Bq)      # Wq frozen; only Aq, Bq get blame
# (c) train nothing (zero-shot)          (the floor)
```

The experiment: take your Week-9-trained GPT, adapt it to a genuinely different text (train on prose? adapt to poetry / another book). Report: new-domain validation loss for (a)/(b)/(c), AND trainable-parameter counts. The headline number: what % of full-fine-tune's improvement did LoRA capture while training what % of the weights? (Literature at scale: ~95%+ of quality for <1% of weights. Your scale will be noisier — measure, don't assume.)

Week 11 exercises

1. Rank sweep: LoRA $r = 1 / 4 / 16$. Quality vs adapter size table. Where do returns die at your scale?
2. Stack the campaigns: your distilled student (W10) + 4-bit (W11) + pruned 30% (M1). Total shrink vs your Month-2 teacher, and total quality cost. This one table is your track's thesis in numbers.
3. Explain in your log, three sentences, why LoRA's memory saving during TRAINING is even bigger than its parameter saving. (Hint: what does momentum store per trainable weight?)

ANSWERS

1: typically $r=4$ -ish suffices for small domain shifts; $r=1$ too cramped; $r=16$ wasteful at this scale. **2:** commonly 5-10x smaller within a few points of teacher quality — YOUR numbers are the deliverable. **3:** optimizer state (momentum etc.) is stored per TRAINABLE weight — freeze 99%, and that whole overhead vanishes with it.

WEEK 12 — Capstone: Your Domain Model, Under Budget

Goal: everything, combined, on purpose: a small model for ONE domain you care about, built under an explicit compute budget, written up like research. This is the miniature of your long-term ambition.

12.1 The brief (constraints are the design)

- **Pick a domain** with 200k+ characters of text you can assemble: your field's documentation, a genre of stories, your own writing, recipes — something whose quality YOU can judge by reading.
- **Set a budget BEFORE starting:** total training multiplies $\leq 2\times$ your Month-2 best run (use your ai_9 accounting). Every choice must be justified against it.
- **Deliverable:** the best next-char model you can build for that domain within budget — plus the write-up.

12.2 The playbook (choose, don't just apply everything)

```
1. CURATE (W9) : clean the domain corpus; run the three-way control if time allows
2. DECIDE size : your scaling plot (W8) says what the budget can afford -- use it to CHOOSE D
3. TRAIN base : Month-2 recipe; overfit-one-batch sanity check first
4. TEACH (W10) : if your bigger Month-2 GPT is decent on this domain, distill from it
5. ADAPT (W11) : if a related base exists, LoRA may beat training fresh -- justify either way
6. SHRINK (W11+M1): quantize + prune the final artifact; report size vs quality
7. MEASURE always: validation loss, verbatim-recall rate (your W9 metric), samples, total multiplies spent
```

The skill being tested is not applying all seven — it's **choosing** with reasons. "I skipped distillation because my teacher was weak on this domain (val loss X)" is a research decision, stated like one.

12.3 The write-up: a real miniature paper

1. **Goal & budget** — domain, why, the multiply budget, success criterion (stated in advance!).
2. **Method** — which levers you chose and why; which you skipped and why.
3. **Results** — the table: model size, bits, training multiplies, val loss, recall-rate; plus your favorite and most embarrassing generated samples (both!).
4. **What I'd try with 10x budget** — three ideas, each in one sentence with the mechanism you expect.
5. **Limits** — tiny scale, one domain, character-level; say it plainly. Honest limits raise credibility, never lower it.

12.4 Day 90, and the door it opens

Repo tagged v1.0. Read your Day-1 XOR file, then your capstone write-up, in one sitting. That distance — from hand-tracing a two-weight nudge to running budget-constrained distillation experiments with honest controls — is what three months of concept → piece → commit builds.

What comes after (the Month-4+ horizon, so the path never fogs): word/subword tokens instead of characters (BPE), autograd frameworks (PyTorch — you've EARNED the abstraction now, having built everything under it once), GPU training, nanoGPT at real scale for your raw-vs-clean experiment with more compute, and reading your first papers cold: start with the Chinchilla scaling paper and the original distillation paper — you now personally own every concept they use.

- ☐ Week 9: my three-way data experiment (raw / clean / size-matched) is measured and logged
- ☐ Week 10: my half-size student beat its hard-label twin, and I can hand-compute the KL that trained it
- ☐ Week 11: my sensitivity table says which parts of MY transformer tolerate 4-bit; my LoRA adapted a domain for a fraction of the weights
- ☐ Week 12: my domain model shipped under budget, with a write-up whose limits section I'm proud of

Codebase complete through: ai_15_data.py → ai_16_distill.py → ai_17_lora.py → capstone/ + writeup.md, tagged v1.0.